T5A2: Least-squares fit with a polynomial

The data are $f_i = f(x_i)$ for $i = 0, 1, \ldots, n$. I approximate them with the polynomial

$$p_m(x) = \sum_{k=0}^{m} c_k x^k,$$

and I look for the coefficients $c_0, c_1, \ldots, c_m$ that make

$$\chi^2 = \sum_{i=0}^{n} [p_m(x_i) - f_i]^2$$

as small as possible.

1) Analytical formulas for the coefficients
χ^2 is a function of the $m+1$ coefficients. At the minimum, all its partial derivatives are zero:

$$\frac{\partial \chi^2}{\partial c_l} = 0, \qquad l = 0, 1, \ldots, m.$$

First I need the derivative of $p_m(x_i) = c_0 + c_1 x_i + \cdots + c_m x_i^m$. Only the term $c_l x_i^l$ depends on c_l, so

$$\frac{\partial p_m(x_i)}{\partial c_l} = x_i^l.$$

With the chain rule,

$$\frac{\partial \chi^2}{\partial c_l} = \sum_{i=0}^{n} 2\,[p_m(x_i) - f_i]\, x_i^l = 0.$$

Next I divide by 2 and put in $p_m(x_i) = \sum_k c_k x_i^k$:

$$\sum_{i=0}^{n} \sum_{k=0}^{m} c_k\, x_i^k\, x_i^l = \sum_{i=0}^{n} f_i\, x_i^l.$$

Both sums are finite, so I can swap them. c_k does not depend on i, so it comes out of the sum over i:

$$\sum_{k=0}^{m} \left(\sum_{i=0}^{n} x_i^{l+k} \right) c_k = \sum_{i=0}^{n} f_i\, x_i^l, \qquad l = 0, 1, \ldots, m.$$

These sums only depend on the data, so I give them names:

$$s_j = \sum_{i=0}^{n} x_i^j, \quad j = 0, 1, \ldots, 2m, \qquad\qquad t_l = \sum_{i=0}^{n} f_i\, x_i^l, \quad l = 0, 1, \ldots, m.$$

$l+k$ goes up to $2m$, so I need s_j up to $j=2m$. Note that $s_0 = n+1$, because $x_i^0 = 1$. Then the equations are

$$\sum_{k=0}^{m} s_{l+k}\, c_k = t_l, \qquad l = 0, 1, \ldots, m.$$

These are $m+1$ linear equations for the $m+1$ unknowns c_k. In matrix form, $Sc = t$:

$$\begin{pmatrix} s_0 & s_1 & \cdots & s_m \\ s_1 & s_2 & \cdots & s_{m+1} \\ \vdots & \vdots & \ddots & \vdots \\ s_m & s_{m+1} & \cdots & s_{2m} \end{pmatrix} \begin{pmatrix} c_0 \\ c_1 \\ \vdots \\ c_m \end{pmatrix} = \begin{pmatrix} t_0 \\ t_1 \\ \vdots \\ t_m \end{pmatrix},$$

so $S_{lk} = s_{l+k}$. S is symmetric, because $s_{l+k} = s_{k+l}$.

If $\det S \neq 0$, Cramer's rule gives all the coefficients:

$$c_k = \frac{\det S^{(k)}}{\det S}, \qquad k = 0, 1, \ldots, m,$$

where $S^{(k)}$ is S with column k replaced by t:

$$S^{(k)} = \begin{pmatrix} s_0 & \cdots & s_{k-1} & t_0 & s_{k+1} & \cdots & s_m \\ s_1 & \cdots & s_k & t_1 & s_{k+2} & \cdots & s_{m+1} \\ \vdots & & \vdots & \vdots & \vdots & & \vdots \\ s_m & \cdots & s_{m+k-1} & t_m & s_{m+k+1} & \cdots & s_{2m} \end{pmatrix}.$$

This is the same as $c = S^{-1}t$.

Cramer's rule needs $\det S \neq 0$. I show that this is true if at least $m+1$ of the x_i are different (so $n \geq m$). For any vector $v = (v_0, v_1, \ldots, v_m)$, I use $s_{l+k} = \sum_i x_i^l x_i^k$ and write $q(x) = \sum_{k=0}^m v_k x^k$:

$$v^T S v = \sum_{l=0}^{m} \sum_{k=0}^{m} v_l\, s_{l+k}\, v_k = \sum_{i=0}^{n} \left(\sum_{l=0}^{m} v_l x_i^l \right) \left(\sum_{k=0}^{m} v_k x_i^k \right) = \sum_{i=0}^{n} q(x_i)^2 \geq 0.$$

If $v \neq 0$, q is not the zero polynomial and its degree is at most m, so it has at most m roots. Then $q(x_i) \neq 0$ for at least one of the $m+1$ different x_i, so $v^T S v > 0$. Therefore, S is positive definite. If $\det S = 0$, there is a $v \neq 0$ with $Sv = 0$, so $v^T S v = 0$. That is not possible, so $\det S \neq 0$ and there is exactly one solution.

The solution is also a minimum of χ^2, not a maximum or a saddle point. I change the coefficients from the solution c_k to $c_k + \delta_k$. Then $p_m(x_i)$ changes by $q(x_i) = \sum_k \delta_k x_i^k$, and with $(a+b)^2 = a^2 + 2ab + b^2$:

$$\begin{aligned} \chi^2(c+\delta) &= \sum_{i=0}^{n} \left[p_m(x_i) - f_i + q(x_i) \right]^2 \\ &= \sum_{i=0}^{n} \left[p_m(x_i) - f_i \right]^2 + \sum_{l=0}^{m} \delta_l \left(\sum_{i=0}^{n} 2 \left[p_m(x_i) - f_i \right] x_i^l \right) + \sum_{i=0}^{n} q(x_i)^2 \\ &= \chi^2(c) + \sum_{l=0}^{m} \delta_l \frac{\partial \chi^2}{\partial c_l} + \delta^T S\, \delta. \end{aligned}$$

The middle term is zero, because $\partial\chi^2/\partial c_l = 0$ at the solution. The last term is $\delta^T S \delta > 0$ for $\delta \neq 0$. Therefore, $\chi^2(c+\delta) > \chi^2(c)$ for every change δ, so the solution is the minimum of χ^2.

Check with $m = 1$, a straight line. Then

$$S = \begin{pmatrix} n+1 & s_1 \\ s_1 & s_2 \end{pmatrix}, \qquad \det S = (n+1)s_2 - s_1^2,$$

and Cramer's rule gives

$$c_0 = \frac{1}{\det S} \det \begin{pmatrix} t_0 & s_1 \\ t_1 & s_2 \end{pmatrix} = \frac{s_2 t_0 - s_1 t_1}{(n+1)s_2 - s_1^2},$$

$$c_1 = \frac{1}{\det S} \det \begin{pmatrix} n+1 & t_0 \\ s_1 & t_1 \end{pmatrix} = \frac{(n+1)t_1 - s_1 t_0}{(n+1)s_2 - s_1^2}.$$

This is the linear fit in section 2.2 of the book (Eqs. 2.29 and 2.30, with the top and bottom multiplied by -1).

2) Main steps of the numerical algorithm
Numerically I do not use Cramer's rule. One determinant costs about as much as one Gaussian elimination, so Cramer's rule needs $m + 2$ eliminations, while solving $Sc = t$ directly needs only one.

Step 1: Input the data x_i, f_i for $i = 0, 1, \ldots, n$ and the order m. At least $m + 1$ of the x_i must be different, so that $\det S \neq 0$ (see 1)).

Step 2: Compute the sums s_j for $j = 0, 1, \ldots, 2m$ and t_l for $l = 0, 1, \ldots, m$, in one loop over the data points. For each point I get the powers by multiplying with x_i every time, $x_i^{j+1} = x_i^j \cdot x_i$, so I do not need a power function.

Step 3: Fill the $(m + 1) \times (m + 1)$ matrix $S_{lk} = s_{l+k}$ and the vector t.

Step 4: Solve $Sc = t$ with Gaussian elimination with partial pivoting (section 5.3 in the book). For each column j, I swap row j with the row (from j down) that has the largest $|S_{rj}|$, and I swap the same two elements of t. This way I never divide by a zero or very small pivot. Then I subtract S_{rj}/S_{jj} times row j from every row r below it (also in t), so the elements below the diagonal become zero. Now S is upper triangular, and I find the coefficients by back substitution, from c_m up to c_0:

$$c_l = \frac{1}{S_{ll}} \left(t_l - \sum_{k=l+1}^{m} S_{lk}\, c_k \right),$$

with S and t after the elimination.

Step 5: Output $c_0, c_1, \ldots, c_m$.

3) Subprogram
Schematically, the subprogram for steps 1 to 5 and the Gaussian elimination it calls are:

```
subprogram leastSquares(x[0..n], f[0..n], m) returns c[0..m]
    #Step 2: the sums s_j and t_l
    s[0..2m] = 0
    t[0..m] = 0
    for i = 0 to n
        xp = 1                          #xp = x_i^j, starts with x_i^0 = 1
        for j = 0 to 2m
            s[j] = s[j] + xp
            if j <= m: t[j] = t[j] + f[i]*xp
            xp = xp*x[i]                #next power of x_i
    #Step 3: fill S
    for l = 0 to m
        for k = 0 to m
            S[l][k] = s[l+k]
    #Step 4: solve S c = t
    c = gauss(S, t)
    #Step 5
    return c

subprogram gauss(A[0..N-1][0..N-1], b[0..N-1]) returns x[0..N-1]
    #elimination with partial pivoting
    for j = 0 to N-2
        piv = the row r >= j with the largest |A[r][j]|
        swap rows j and piv of A, and b[j] and b[piv]
        for r = j+1 to N-1
            fac = A[r][j]/A[j][j]
            for k = j to N-1
                A[r][k] = A[r][k] - fac*A[j][k]
            b[r] = b[r] - fac*b[j]
    #back substitution
    for r = N-1 down to 0
        x[r] = (b[r] - sum of A[r][k]*x[k] for k = r+1 to N-1)/A[r][r]
    return x
```

Here $N = m + 1$.